

DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Diseño Jerárquico

Bloques de construcción digitales

Sistemas Digitales

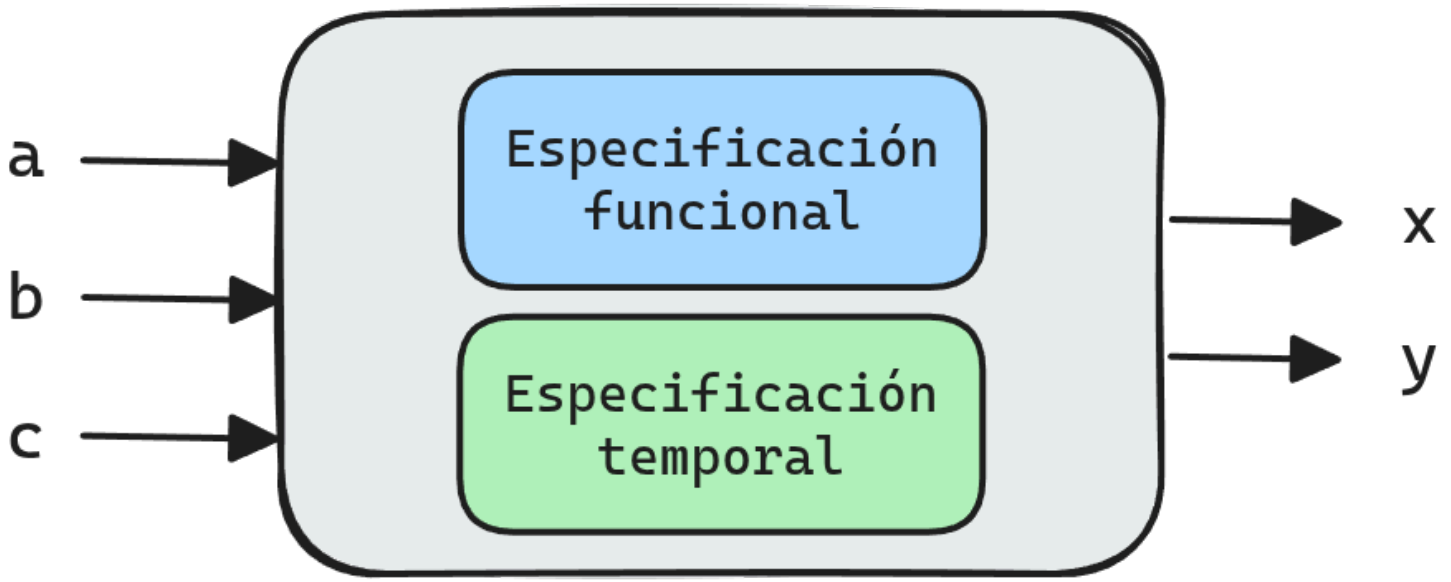
2do Cuatrimestre 2026

FCEN - UBA

Jerarquía, Modularidad y Regularidad

Subiendo por la jerarquía

- De **compuertas** / **muxes** / **decoders** a bloques más grandes: comparadores, shifters, contadores, memorias...
- Cada bloque tiene una interfaz **bien definida** y se puede tratar como caja negra



- Usaremos estos bloques en la clase de microarquitectura para armar nuestro micro

Complejidad creciente

Para manejar la complejidad creciente de estos sistemas, los diseñadores usan tres principios de diseño:

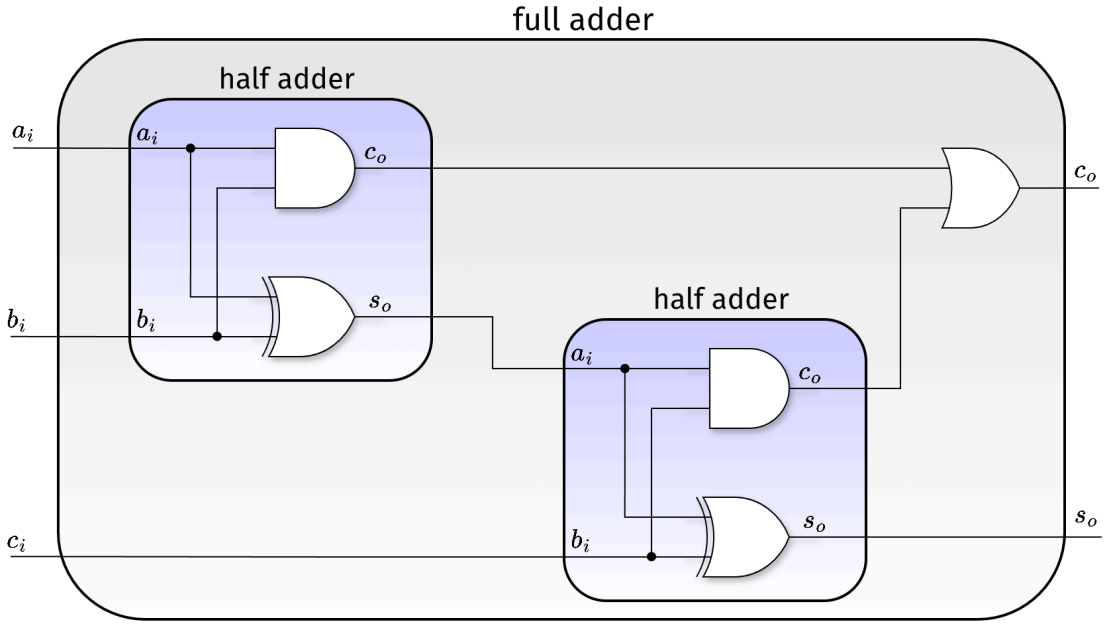
- **Jerarquía**
- **Modularidad**
- **Regularidad**

Estos principios se aplican tanto a sistemas de software como de hardware.

Jerarquía

> ***Jerarquía:*** *involucra dividir un sistema en módulos, y luego subdividir cada uno de estos módulos hasta que las piezas sean fáciles de entender.*

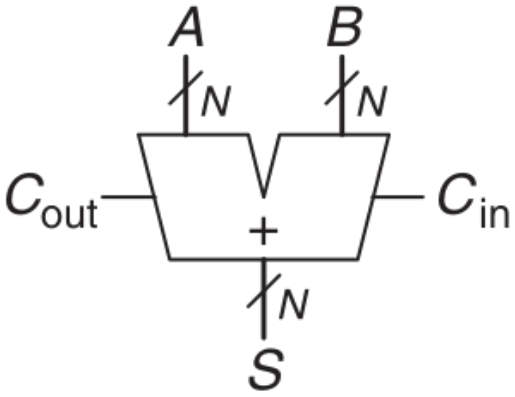
- Dividir un sistema complejo en submódulos
- Cada nivel esconde el detalle del nivel inferior (*abstracción*)
- ***Ejemplo***: *un* ***full adder*** *se construye con 2* ***half adders*** *y 1* ***OR gate***



Modularidad

> ***Modularidad:*** *dicta que los módulos tienen funciones e interfaces bien definidas, de modo que se conectan fácilmente sin efectos secundarios inesperados.*

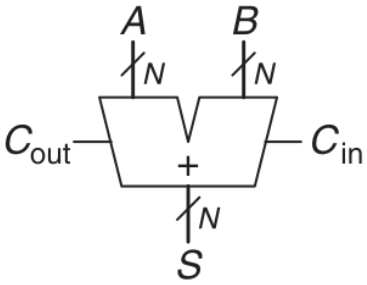
- Módulos con contratos bien definidos, reusables como *caja negra*
- Se conectan sin efectos secundarios inesperados (sin *side effects*)
- La implementación puede cambiar mientras que el contrato se mantenga
- ***Ejemplo***: *el* ***full adder*** *tiene entradas A, B y Cin, y salidas S y Cout. La implementación interna puede cambiar mientras que la interfaz se mantenga.*

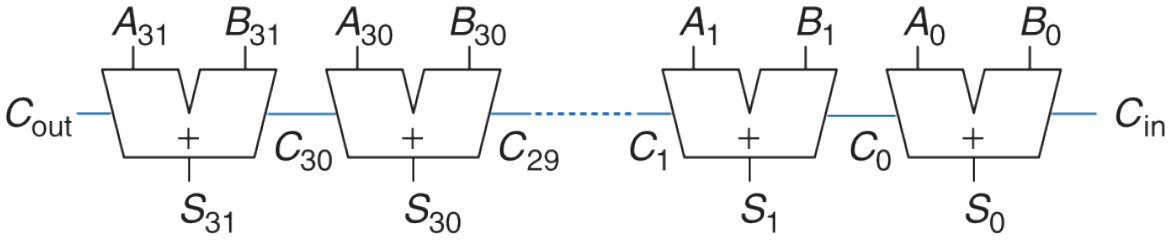


Regularidad

> ***Regularidad:*** *busca uniformidad entre módulos. Los módulos comunes se reutilizan muchas veces, reduciendo el número de módulos distintos que deben diseñarse.*

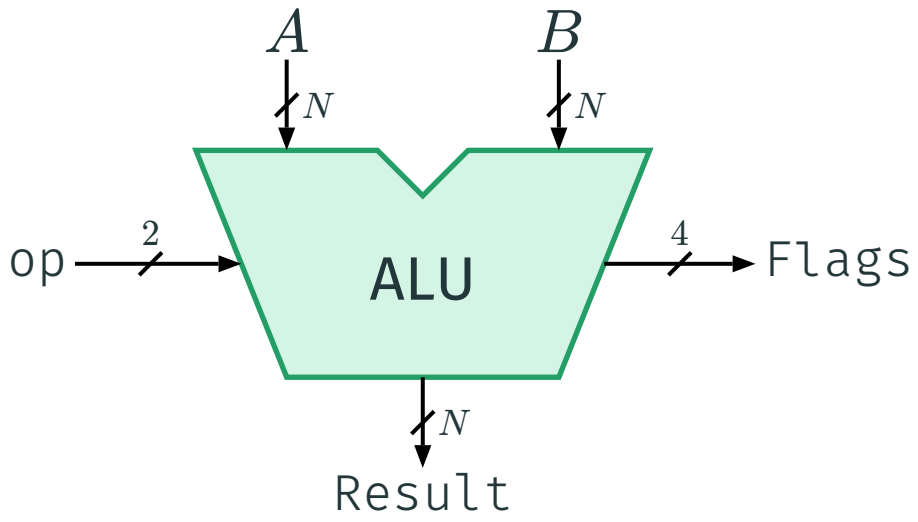
- Estructura uniforme entre instancias del módulo
- Diseñar módulos útiles que puedan ser usados en diferentes contextos
- Se buscan módulos parametrizables (ej: ancho de bits (N genérico))
- ***Ejemplo***: *el* ***full adder*** *se reusa N veces en un* ***adder de N bits***





Módulo de ejemplo: ALU

Ejemplo: ALU de N bits y cuatro operaciones



op	Result
00	$A + B$
01	$A - B$
10	$A \wedge B$
11	$A \vee B$

Flag	Nombre	Definición	Implentación típica
N	*Negative*	El resultado es negativo	MSB del resultado: `Result[N-1]`
Z	*Zero*	El resultado es cero	`Result` $=$ `0` (típicamente NOR de todos los bits)
C	*Carry*	Acarreo	C_{out} del sumador
V	*oVerflow*	El resultado supera los límites representables	e.g. comparación de signos de operandos y resultado

C y V solo tienen sentido en `op` aritméticas; en `op` lógicas valen 0.

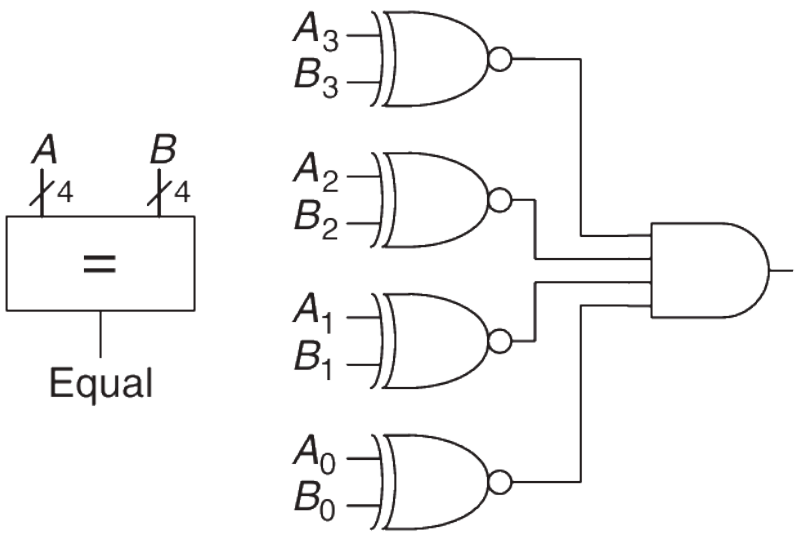
La resta se puede hacer con el mismo sumador: $A - B = A + \overline{B} + 1$. No es recomendable por complejidades aritméticas (e.g. A-0) y sistémicas (cadena combinacional muy larga).

Comparadores

¿Qué es un comparador?

- Un **comparador** determina la relación entre dos números: si son iguales, si el primer número es mayor o si es menor que el otro
- Recibe dos números de largo fijo (N bits), A y B, y devuelve resultados de comparación en una o más salidas de 1 bit
- **Comparador de igualdad**: indica si $A = B$, se informa con una sola salida
- **Comparador de magnitud**: indican el valor relativo entre A y B, se informa con una o más salidas.

Comparador de igualdad



- Compara bit a bit con **XNOR**: la columna i da 1 sii $A_i = B_i$
- Un **AND** final: son iguales sii **todas** las columnas lo son

$$\text{Equal} = \prod_{i=0}^{N-1} \overline{A_i \oplus B_i}$$

Comparador de magnitud (sin signo)

- Para comparar dos números sin signo ($A < B$), se puede hacer resolviendo la resta entre ambos: $A - B$
- El resultado de la resta puede ser:
  - positivo o cero: por lo que $A \geq B$
  - negativo: en cuyo caso estamos ante $A < B$

¿Cómo detectamos esto?

- El MSB del resultado **no** es un signo (estamos en notación sin signo), así que no sirve para comparar
- $A < B$ exactamente cuando la resta **genera borrow**:

$$A < B \Leftrightarrow B_{\text{out}} = 1$$

Ej. (3 bits): `101 - 001 = 100` (5 - 1 = 4) $B_{\text{out}} = 0 \rightarrow A \geq B$ ✓
Ej. (3 bits): `001 - 110 = 011` (1 - 6 = -5 ≡ 3) $B_{\text{out}} = 1 \rightarrow A < B$ ✓

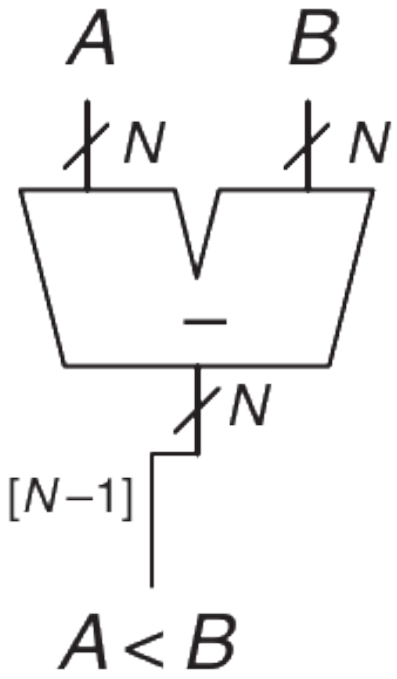
Si el MSB fuera un signo, diría lo contrario en los dos casos: el que decide es el borrow.

Comparación sin signo (2 bits)

A		B		$A-B$		C_{out}	B_{out}	$A<B$
dec	bin	dec	bin	dec	bin			
0	00	0	00	0	00	1	0	0
0	00	1	01	3	11	0	1	1
0	00	2	10	2	10	0	1	1
0	00	3	11	1	01	0	1	1
1	01	0	00	1	01	1	0	0
1	01	1	01	0	00	1	0	0
1	01	2	10	3	11	0	1	1
1	01	3	11	2	10	0	1	1
2	10	0	00	2	10	1	0	0
2	10	1	01	1	01	1	0	0
2	10	2	10	0	00	1	0	0
2	10	3	11	3	11	0	1	1
3	11	0	00	3	11	1	0	0
3	11	1	01	2	10	1	0	0
3	11	2	10	1	01	1	0	0
3	11	3	11	0	00	1	0	0

C_{out} es el acarreo de $A + \overline{B} + 1$. $B_{\text{out}} = \overline{C_{\text{out}}}$ y coincide con $A < B$ (filas resaltadas).

Comparador de magnitud (con signo)



- Para comparar dos números con signo ($A < B$), también se puede resolver usando la resta entre ambos: $A - B$
- El MSB del resultado **sí** es un signo, si es negativo (1), $A < B$. Si no, $A \geq B$
- Esta estrategia funciona siempre y cuando el resultado no se vaya de la representación de N bits (overflow)

> No alcanza la resta y el signo si hay overflow

¿Por qué falla?

- Cuando la resta $A - B$ genera **overflow**: el resultado no entra en N bits

Funciona bien (4 bits):

A = `1111` (-1), B = `0001` (1)

$A - B = 1110$ (-2)

signo = 1 → dice $A < B$ ✓

Falla (4 bits):

A = `1000` (-8), B = `0001` (1)

$A - B = 0111$ (7, con overflow)

signo = 0 → dice $A \geq B$ ×

En estrategia de la **resta**: falla cuando A y B tienen signos distintos y el resultado cambia de signo respecto de A — eso es justamente **overflow** ($V = 1$).

Comparación con signo (2 bits)

A		B		$A-B$		$A-B$ (real)	N	V	$A<B$
dec	bin	dec	bin	dec	bin				
0	00	0	00	0	00	0	0	0	0
0	00	1	01	-1	11	-1	1	0	1
0	00	-2	10	-2	10	2	1	1	0
0	00	-1	11	1	01	1	0	0	0
1	01	0	00	1	01	1	0	0	0
1	01	1	01	0	00	0	0	0	0
1	01	-2	10	-1	11	3	1	1	0
1	01	-1	11	-2	10	2	1	1	0
-2	10	0	00	-2	10	-2	1	0	1
-2	10	1	01	1	01	-3	0	1	1
-2	10	-2	10	0	00	0	0	0	0
-2	10	-1	11	-1	11	-1	1	0	1
-1	11	0	00	-1	11	-1	1	0	1
-1	11	1	01	-2	10	-2	1	0	1
-1	11	-2	10	1	01	1	0	0	0
-1	11	-1	11	0	00	0	0	0	0

(real) = $A-B$ sin truncar. Resaltado: overflow ($V=1$) — ahí N no coincide con $A<B$.

Arreglando el comparador

- No alcanza con mirar el signo del resultado (N): hay que combinarlo con el flag de **overflow** (V)
- $A < B$ exactamente cuando $N \oplus V = 1$

Si no hay overflow ($V = 0$), miro N. Si hay overflow, el XOR **invierte** el signo.

Mismo restador, distinta lectura

- El hardware es **el mismo**: $A - B = A + \overline{B} + 1$. Los bits del resultado no cambian
- Lo único que cambia es **qué flag se mira**:

	Sin signo	Con signo
$A < B$	$\overline{C}$	$N \oplus V$

La ALU calcula los flags (N, C, V...) y quien la usa elige cuál mirar según cómo interprete los operandos.

Comparador en SystemVerilog


```sv
module comparador #(
    parameter int N = 4
) (
    input  logic [N-1:0] a, b,
    output logic         eq,   // A == B
    output logic         lt_u, // A <  B, sin signo (borrow)
    output logic         lt_s  // A <  B, con signo (N xor V)
);

    logic [N-1:0] resta;
    logic         bout, n, v;

    assign eq = &(a ~^ b);                          // XNOR + AND
    assign {bout, resta} = {1'b0, a} - {1'b0, b};   // Extensión con ceros
    assign n = resta[N-1];
    assign v = (a[N-1] ^ b[N-1]) & (a[N-1] ^ n);
    assign lt_u = bout;
    assign lt_s = n ^ v;

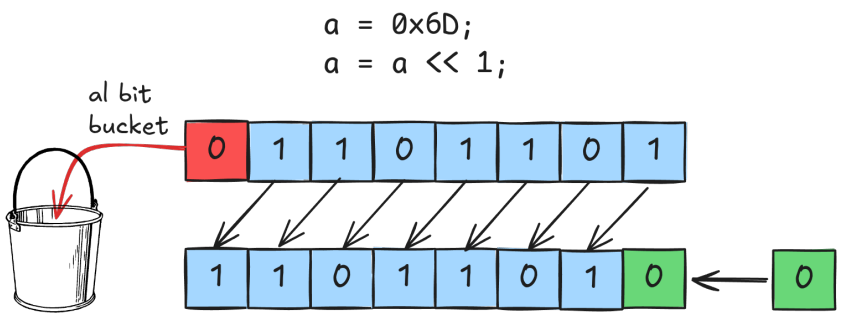
endmodule
```

Desplazadores (*shifters*) y Rotadores

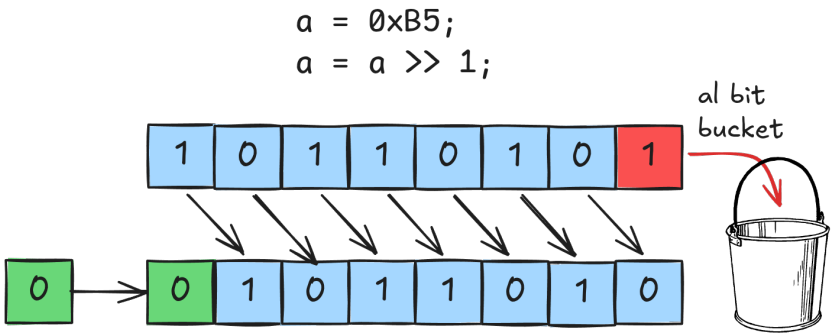
Tipos de desplazamiento: repaso

Desplazamiento lógico

- A izquierda

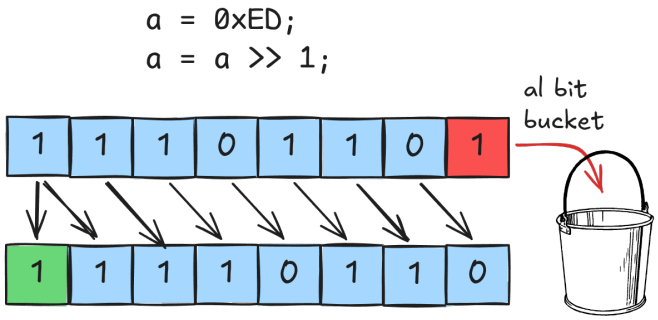


- A derecha

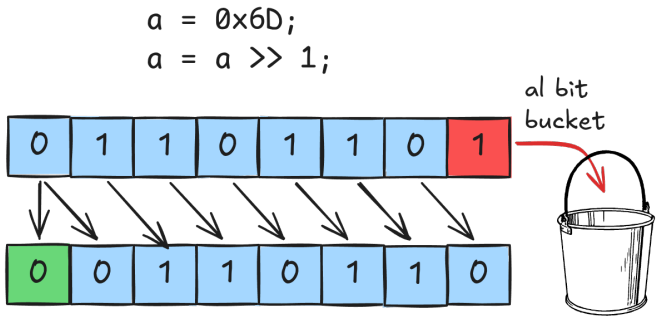


Desplazamiento aritmético

- MSB: 1



- MSB: 0



Tipos de desplazadores

- **Logical shifter**: desplaza y rellena con 0

  `11001 >> 2 = 00110`

  `11001 << 2 = 00100`

- **Arithmetic shifter**: igual, pero en shift derecho rellena con copia del bit de signo (MSB)

  `11001 >>> 2 = 11110`

  `11001 <<< 2 = 00100`

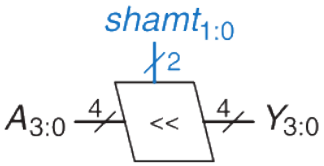
- **Rotador**: los bits que salen por un extremo entran por el otro

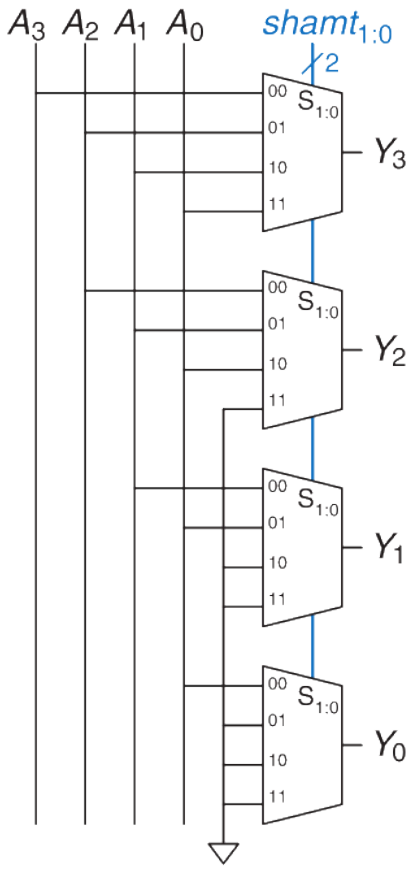
  `11001 ROR 2 = 01110`

  `11001 ROL 2 = 00111`

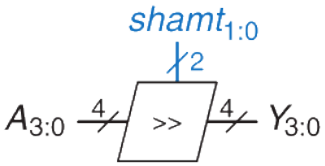
Notación: ROR rotate right, ROL rotate left. Shift lógico: >>, <<. Shift aritmético: >>>, <<<.

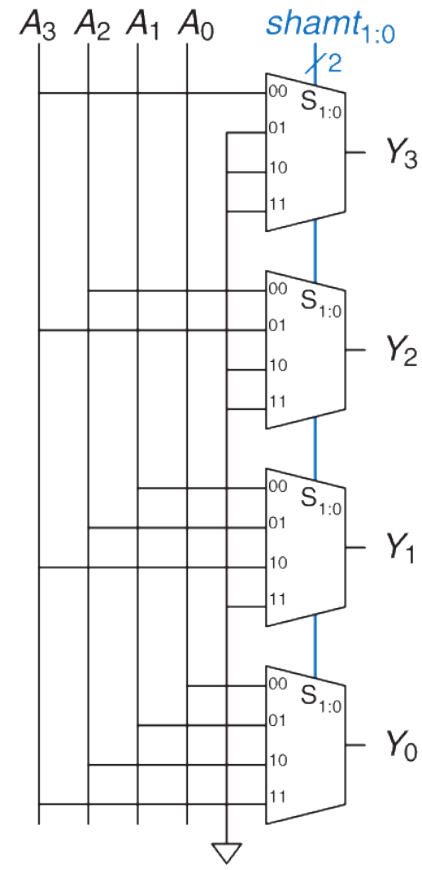
Implementación con muxes



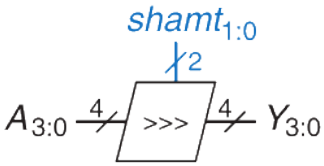


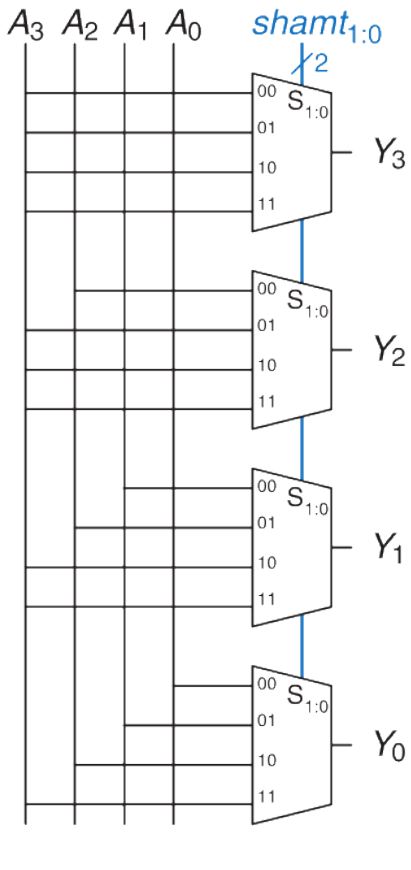
Shift lógico
a izquierda (<<)





Shift lógico
a derecha (>>)





Shift aritmético
a derecha (>>>)

Implementación con muxes (cont.)

Con entrada A y salida Y de N bits:

- Se arma con N muxes N:1: uno por cada bit de salida Y_i
- La entrada `shamt` (*shift amount*, $\log_2 N$ bits) es el selector: elige desplazar de 0 a N-1
- Cuando `shamt=0`, Y=A en los tres casos
- Los tres usan la misma estructura. Lo único que cambia es el **cableado**:
  - Qué bit llega a Y_i si `shamt` $= s$: A_{i-s} (izquierda) o A_{i+s} (derecha)
  - Qué entra cuando no hay bit: `0` en los lógicos, A_{N-1} (el signo) en el aritmético

Shift como multiplicación/división

- Shift **izquierda** por N bits = multiplicar por 2^N

000011_2 << 4 = 110000_2, equivalente a $3_{10} \times 2^4 = 48_{10}$

- Shift **aritmético derecho** por N bits = dividir por 2^N (con signo)

11100_2 >>> 2 = 11111_2, equivalente a $-4_{10} / 2^2 = -1_{10}$

Shifters en SystemVerilog


```sv
// Shifter de N bits
module shifter #(
    parameter int N = 4
) (
    input  logic [N-1:0]          a,
    input  logic [$clog2(N)-1:0] shamt,
    output logic [N-1:0]          sll,  // A << shamt  (lógico, izquierda)
    output logic [N-1:0]          srl,  // A >> shamt  (lógico, derecha)
    output logic [N-1:0]          sra   // A >>> shamt (aritmético, derecha)
);

    assign sll = a << shamt;
    assign srl = a >> shamt;
    assign sra = $signed(a) >>> shamt;

endmodule
```

`>>>` solo es aritmético si el operando es `signed`: por eso el `$signed(a)`.

Registros de desplazamiento

Shift register

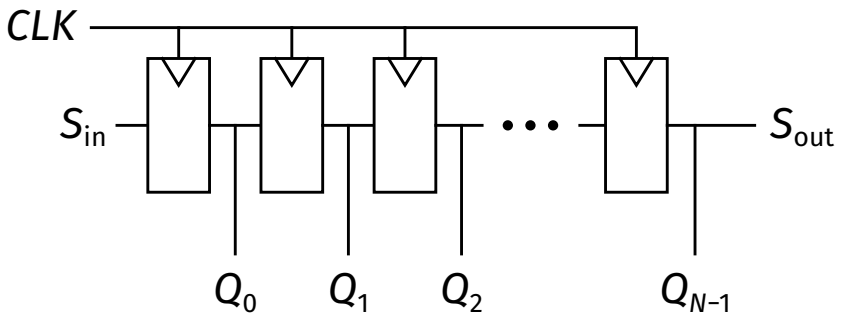
Motivación:

¿Cómo hacemos para leer un entero de 64 bits desde un puerto USB?

> USB viene de *Universal Serial Bus* y el término serial indica que manda los bits en forma serie, i.e. uno a la vez.

Con un **shift register**:

- Entrada serie (Sin), salida serie (Sout), N salidas paralelas
- N flip-flops en cadena; en cada flanco entra un bit y se desplaza el resto



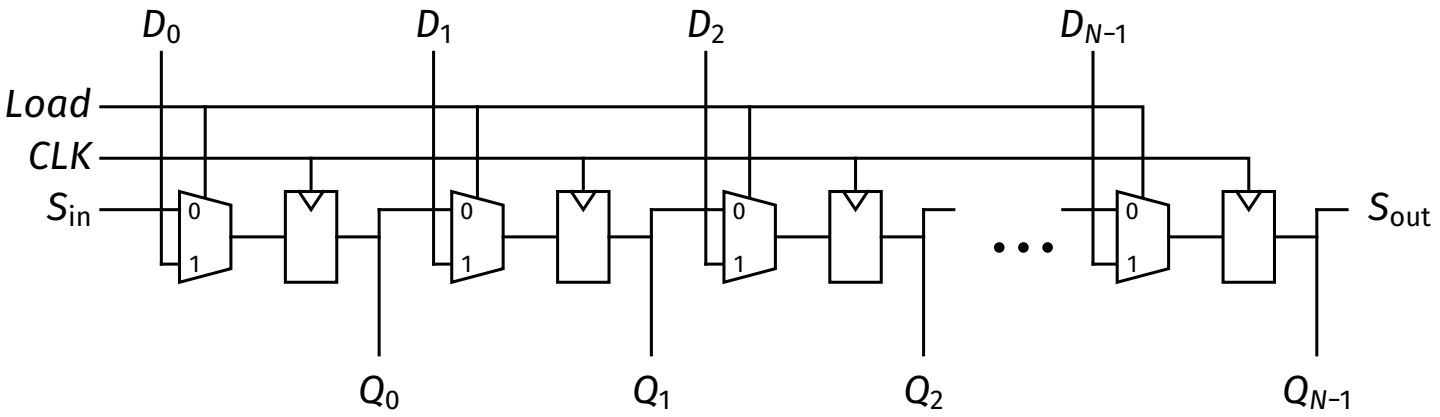
Aplicaciones:

- Conversión serie paralelo

Shift register con carga paralela

Podemos extender el shift register con:

- Entrada paralela D + señal Load



Aplicaciones:

- Conversión paralelo serie y serie paralelo (con el mismo módulo)
- Generación de patrones pseudoaleatorios: Precargando los flip-flops con un valor inicial (semilla) y realimentando alguna función de las salidas Q_n en la entrada.
- Debug de secuenciales con *scan chains* (advanced).

Shift register: implementación


```sv
module shiftreg #(parameter N = 8)
    (input logic          clk,
     input logic          reset, load,
     input logic          sin,
     input logic  [N-1:0] d,
     output logic [N-1:0] q,
     output logic         sout);

always_ff @(posedge clk, posedge reset)
    if (reset)      q <= 0;
    else if (load)  q <= d;
    else            q <= {q[N-2:0], sin};

assign sout = q[N-1];

endmodule
```

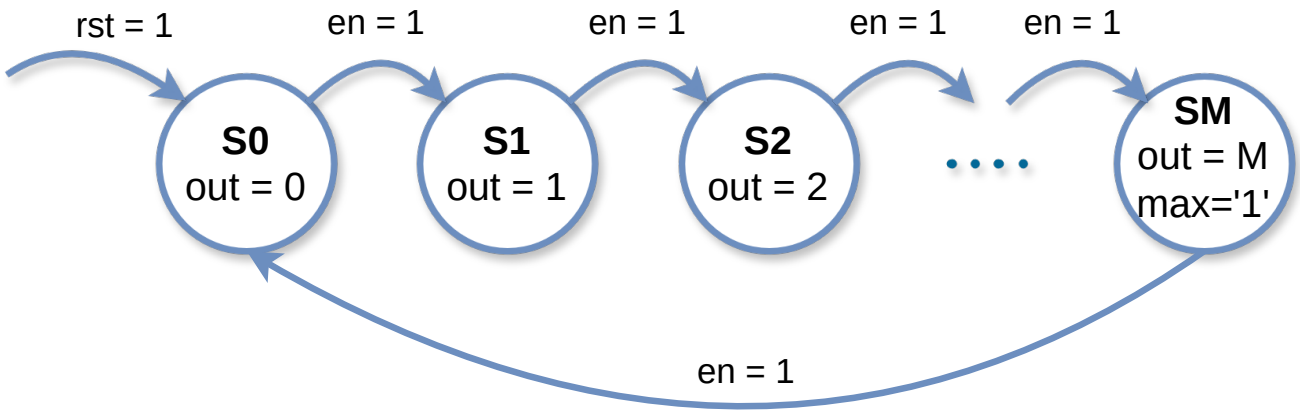
Contadores

Contador binario

Es un sistema secuencial con clock y reset: **una FSM**.

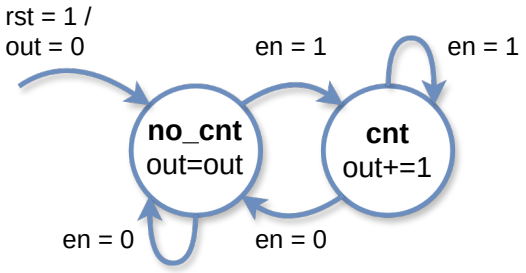
- Entradas: `clk`, `rst` y `en` (habilitación: si `en = 0`, no cuenta)
- Salidas: `out` (la cuenta, N bits) y `max` (vale 1 cuando $\text{out} = M$)
- Características:
  - Registro de estado de N bits $\rightarrow$ cuenta de 0 a $M = 2^N - 1$
  - Vuelve a 0 al llegar a 2^N (circular)

¡Listo! Ya sabemos diseñarlo: 😵



Contador binario (cont.)

Los estados $S_0 ... S_M$ solo difieren en el valor de `out`: podemos colapsarlos.



La FSM solo decide **si cuenta o no** (`cnt` / `no_cnt`); la cuenta queda guardada en `out`.

¿Qué falta?

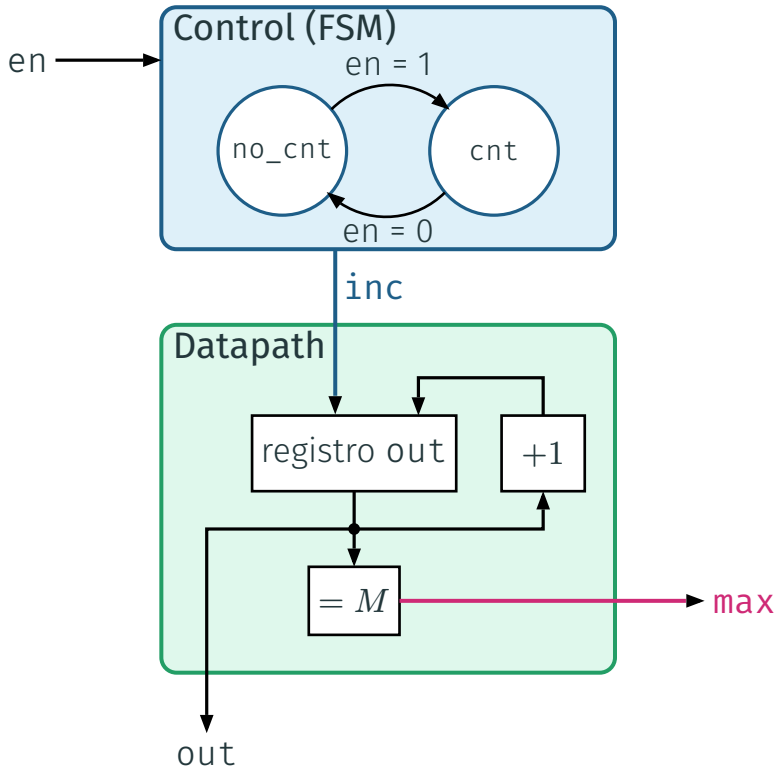
- `max`: $f(\text{out}) = (\text{out} = M) \rightarrow$ **Lógica de salida**
- `out = 0` luego del máximo: $f(\text{out})$: si $\text{out} = M$, sigue 0 $\rightarrow$ **Lógica de estado siguiente**

Todo esto es lógica combinacional 😎

Contador: FSM + datapath

Separamos el contador en dos partes que se comunican:

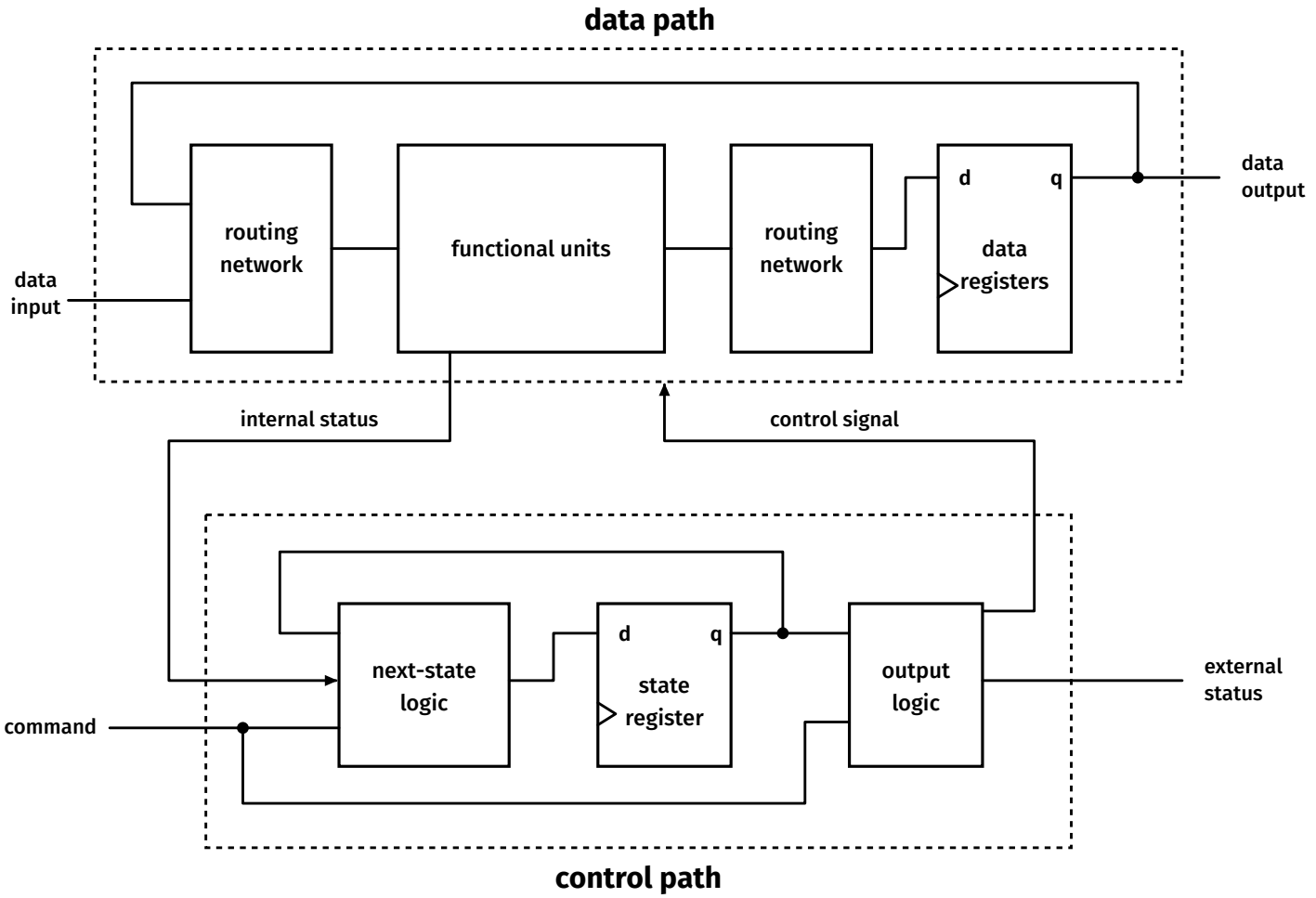
- **Control** (FSM): decide *qué hacer* → 2 estados: `cnt` / `no_cnt`
- **Datapath**: guarda y opera los datos → registro `out`, sumador $+1$, comparador $\text{out} = M$
- Control → datapath: **señales de control** (`inc`: "sumá 1")
- Datapath → afuera / control: **señales de estado** (`max`)



Esto se llama **FSM con *datapath* (FSMD)**: 2 estados + un registro de N bits, en vez de 2^N estados.

FSMD: caso general

FSM con *datapath* (FSMD).



Contador: Implementación con FSM + datapath


```sv
module contador_FSM #(parameter int N = 8)
    (input logic clk,
    input logic reset,
    input logic enable,
    output logic [N-1:0] q,
    output logic max);

    // Estados del diagrama: no_cnt y cnt
    typedef enum logic [0:0] {no_cnt, cnt} state_t;

    state_t state, next_state;
    logic [N-1:0] q_out;
    logic inc;

    // Registro de estado:
    always_ff @(posedge clk, posedge reset)
        if (reset) state <= no_cnt;
        else       state <= next_state;

    // Lógica de estado siguiente:
```

Contador: Implementación con FSM + datapath


```
    always_comb begin
        next_state = state;
        unique case (state)
            no_cnt:
                if (enable) next_state = cnt;
            cnt:
                if (!enable) next_state = no_cnt;
            default: next_state = no_cnt;
        endcase
    end

    // Lógica de salida: controles del datapath + salidas del sistema
    always_comb begin
        max = (q_out == {N{1'b1}}); // 2^N - 1, f(datapath)
        inc = 1'b0;
        unique case (state)
            no_cnt:  inc = 1'b0;
            cnt:     inc = 1'b1;
            default: inc = 1'b0;
        endcase
```

Contador: Implementación con FSM + datapath


```
    end

    // DATAPATH
    // El "out = 0 luego del máximo" es gratis: q_out tiene N bits.
    always_ff @(posedge clk, posedge reset)
        if (reset)    q_out <= '0; // rst = 1 / out = 0
        else if (inc) q_out <= q_out + 1;

    // Conexiones
    assign q = q_out;

    endmodule
```

... pero podemos optimizar más la implementación:

Dos estados pueden ser representados por un bit de control.

Contador: Implementación básica

sv

```sv
module contador_basico #(parameter int N = 8)
    (input logic clk,
    input logic reset,
    input logic enable,
    output logic [N-1:0] q,
    output logic max);

    logic [N-1:0] q_int;

    always_ff @(posedge clk, posedge reset)
        if (reset) q_int <= 0;
        else if (enable)
            q_int <= q_int + 1;
            // Sirve esto?: q_int <= (q_int == 2**N-1) ? 0 : q_int + 1;

    assign q = q_int;
    assign max = (q_int == {N{1'b1}}); // 2^N - 1

endmodule
```

Contador: Implementación programable

sv

```sv
// Contador binario de N bits con valor máximo de cuenta programable
module contador_programable #(parameter int N = 8)
    (input logic clk,
    input logic reset,
    input logic enable,
    input logic [N-1:0] max_value, // valor máximo de cuenta, en [0, 2^N-1]
    output logic [N-1:0] q,
    output logic max);

    logic [N-1:0] q_int;

    always_ff @(posedge clk, posedge reset)
        if (reset) q_int <= 0;
        else if (enable)
            q_int <= (q_int == max_value) ? 0 : q_int + 1;

    assign q = q_int;
    assign max = (q_int == max_value);

endmodule
```

Contador: módulo como parámetro

Podríamos (por comodidad) querer pasar el módulo máximo de cuenta que vamos a usar:

```sv
// Contador programable de módulo máximo M: N = clog2(M) se calcula
// internamente (M >= 2); max_count fija el tope de cuenta en [0, M-1]

module contador_modM #(
    parameter int M = 10 // módulo máximo de cuenta
) (
    input  logic clk,
    input  logic reset,
    input  logic enable,
    input  logic [$clog2(M)-1:0] max_count, // tope de cuenta, en [0, M-1]
    output logic [$clog2(M)-1:0] q,
    output logic max
);

    localparam int N = $clog2(M);

    logic [N-1:0] q_int;
```

Contador: módulo como parámetro


```
    always_ff @(posedge clk, posedge reset)
        if (reset) q_int <= 0;
        else if (enable)
            q_int <= (q_int == max_count) ? 0 : q_int + 1;

    assign q = q_int;
    assign max = (q_int == max_count);

endmodule
```



Aplicación: blinky

El hola mundo de diseño digital:

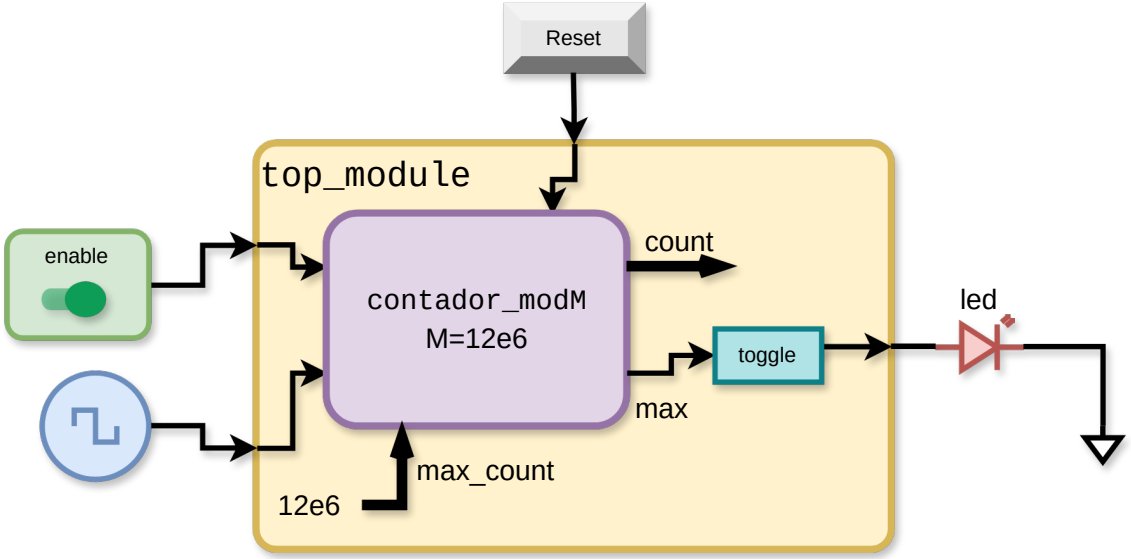
Un led parpadeando cada 1 segundo.

Problema:

- Nuestro clock va a 12 MHz (12 millones de ciclos por segundo).

Solución:

- Un contador que cuente hasta... 12 millones.
- El `max` conmuta el led (la cuenta no nos importa).



Blinky: implementación

sv

```sv
// Blinky: led que conmuta cada M ciclos de clock (default: 12 M ciclos = 1 s @ 12 MHz)

module blinky #(
    parameter int M = 12_000_000 // ciclos de clock por conmutación del led
) (
    input  logic clk,
    input  logic rst,
    input  logic enable,
    output logic led
);

    localparam int N = $clog2(M);
    localparam logic [N-1:0] TOP = N'(M - 1); // cuenta de 0 a M-1

    logic [N-1:0] q;
    // la cuenta no nos importa, solo el pulso max
    logic max;
```

Blinky: implementación


```
    contador_modM #(.M(M)) contador (
        .clk(clk),
        .reset(rst),
        .enable(enable),
        .max_count(TOP),
        .q(q),
        .max(max)
    );

    // Conmuta el led en cada vuelta del contador (una sola vez por vuelta)
    always_ff @(posedge clk, posedge rst)
        if (rst) led <= 0;
        else if (enable && max) led <= ~led;

endmodule
```

Blinky: los tres principios

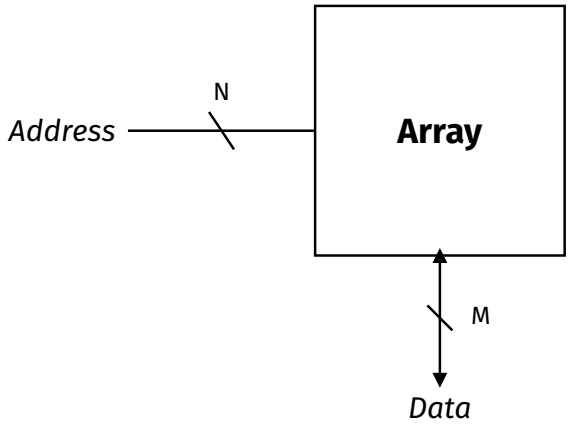
Blinky es chico, pero ya usa los tres principios del inicio:

- **Jerarquía**: `blinky` → `contador_modM` → registro, sumador y comparador → *full adders*, compuertas, LUTs o bloques DSPs. Cada nivel esconde el detalle del de abajo.
- **Modularidad**: `blinky` solo conoce la **interfaz** del contador (`enable`, `max_count` → `max`). Podemos cambiar cómo cuenta por dentro sin tocar `blinky`, mientras la interfaz se mantenga.
- **Regularidad**: es el **mismo** contador, parametrizado. Con `M = 12_000_000` mide un segundo; con `M = 10`, cuenta dígitos decimales.

Memoria

Arrays de memoria: idea general

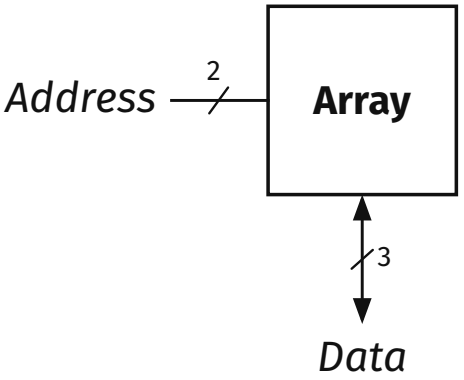
Los *Arrays de Memoria* (o "memorias" coloquialmente) guardan mucha información. Conceptualmente:

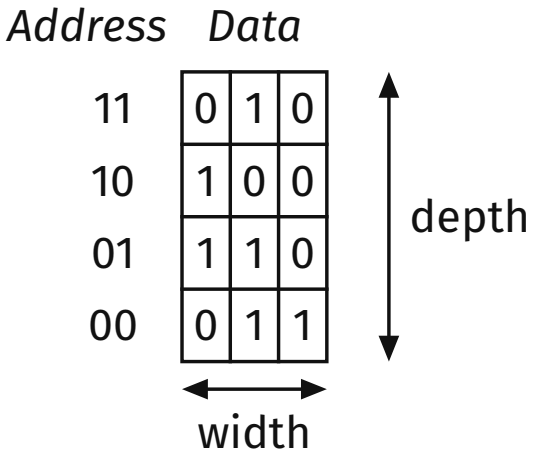


- Arreglo de celdas; *address* (dirección) de N bits selecciona una fila (*word*) de M bits
- Profundidad = 2^N filas, ancho = M bits:
  - $\text{tamaño} = \text{profundidad} \times \text{ancho}$

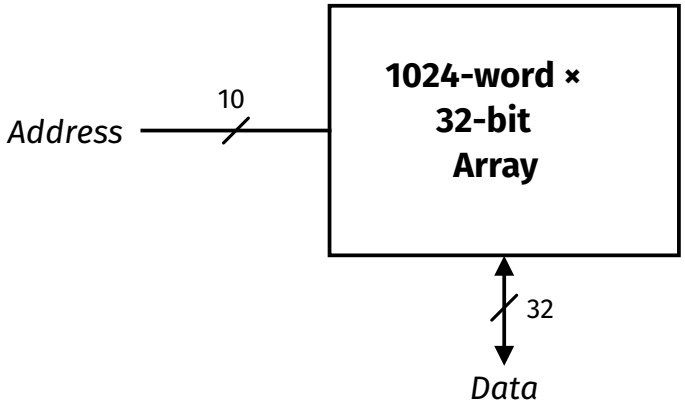
Arrays de memoria: ejemplos

- $N = 2; M = 3$:



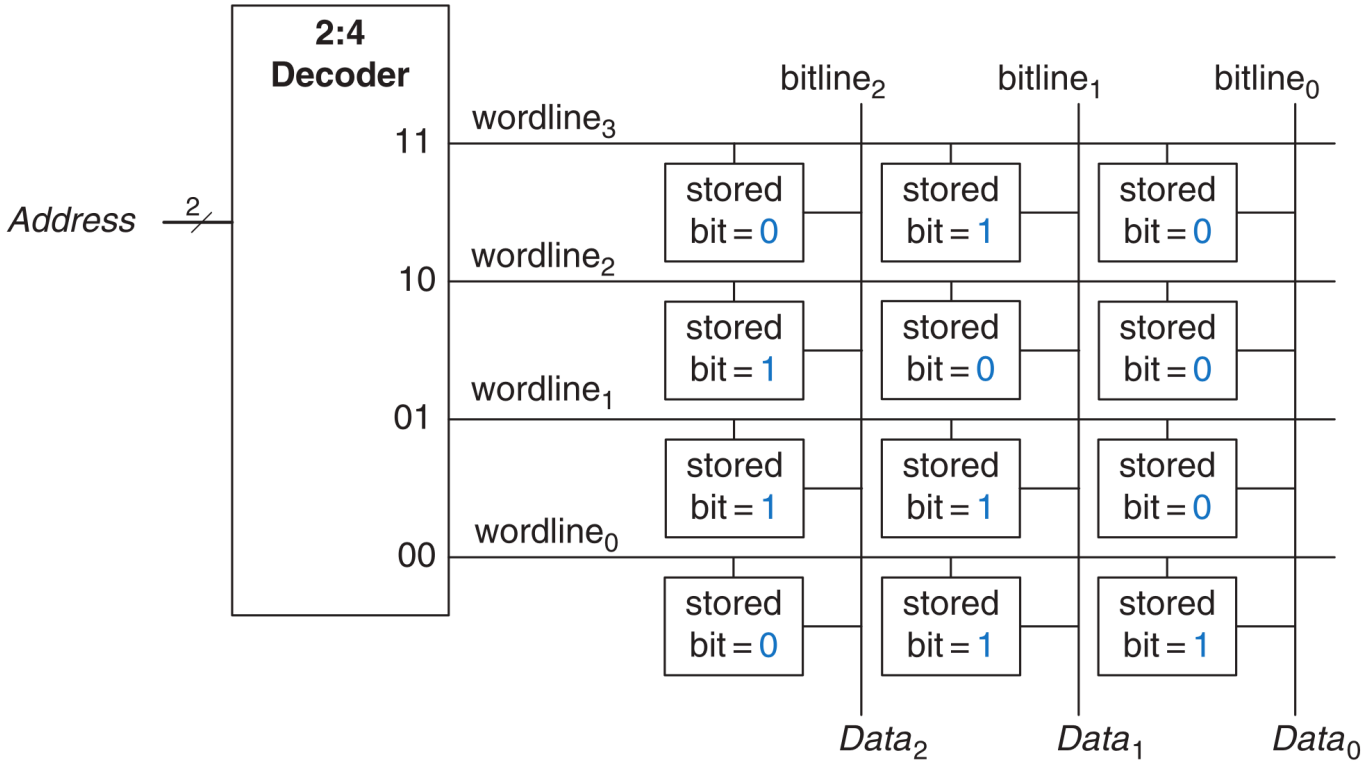


- 1024 posiciones de 32 bits cada una:



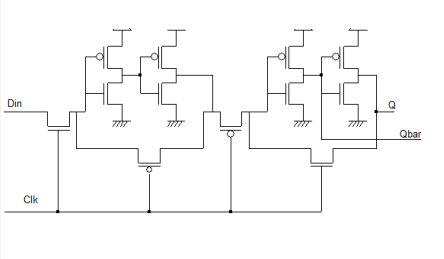
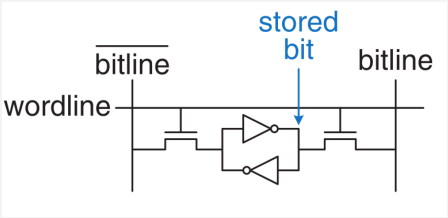
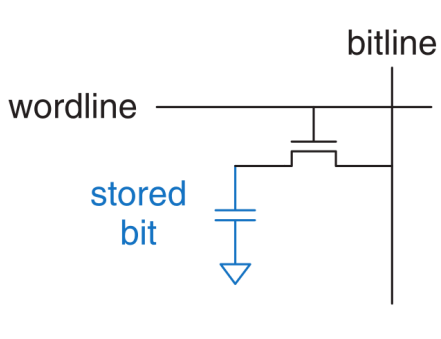
Bit cells

Adentrándonos en la memoria, la unidad básica es el *bit cell*. Una memoria puede ser vista como:



Las *bit cells* están definidas por la tecnología

Tecnologías (se ve en AyOC)

	Flip-flops	SRAM	DRAM
Bit cell			
Transistores	≈ 20	6	1
Latencia	Rápida	Media	Lenta
Usos	Registros	Caché	RAM

Transistores por **bit cell** según Harris & Harris, *Digital Design and Computer Architecture*.

Implementación: RAM básica


```sv
module ram #(parameter N = 6, M = 32)
    (input logic           clk,
     input logic           we,
     input logic  [N-1:0] adr,
     input logic  [M-1:0] din,
     output logic [M-1:0] dout);

logic [M-1:0] mem [2**N-1:0];

always_ff @(posedge clk)
    if (we) mem [adr] <= din;

assign dout = mem[adr];

endmodule
```

¿Preguntas?